VLCBlib_PIC Cookbook

for VLCBlib_PIC release 2.0.0

Ian Hogg and Keith Bruce

October 2026

Table of Contents

1. Introduction **4**
1.1. Location of library 4
1.2. Releases and Versions 4
2. Setting up a Development Environment **5**
2.1. Equipment Required 5
2.1.1. PC 5
2.1.2. PICkit 5
2.1.3. PICkit Adapter cable 5
2.2. Software 5
2.2.1. MPLAB X IDE and MPLAB IPE 5
2.2.2. XC8 compiler 6
2.2.3. Git Tools 6
2.3. Other downloads 6
2.3.1. VLCBlib_PIC 6
2.3.2. VLCB-defs 7
2.3.3. CBUS_PIC_Bootloader 7
3. Module design **8**
3.1. Software Structure 8
3.2. Functionality, NVs and EVs 8
3.3. Services 9
3.4. Event Services 9
3.4.1. Event Teach 9
3.4.2. Event Producer 10
3.4.3. Event Consumer 10
3.5. Memory map 11
3.6. Non Volatile Memory buffers 12
3.6.1. Flash memory 12

3.6.2. EEPROM 12
3.6.3. NV variables 13
4. Project setup in MPLAB X IDE 14
4.1. Create Project folder 14
4.2. Create MPLAB X IDE Project 14
4.3. Import Library files into project 15
4.3.1. Import from VLCB-defs 15
4.3.2. Import necessary files from VLCBlib_PIC 16
4.3.3. Create project files 17
4.3.4. Import project files into project 17
4.4. Project Properties 18
4.5. Updating Project for Different Processors 19
4.6. Compiler output 19
5. Writing module software 21
5.1. Configuring the library in module.h 21
5.1.1. MNS Configuration 21
5.1.2. Additional configuration for NV service 22
5.1.3. Additional configuration for CAN service 23
5.1.4. Additional Configuration for Event Teach service 23
5.1.5. Additional Configuration for Event Producer service 24
5.1.6. Additional Configuration for Event Consumer service 24
5.1.7. Additional configuration for BOOT service 24
5.2. The main program source file 25
5.2.1. Header files 25
5.2.2. Services 25
5.2.3. Transport 25
5.2.4. Callback functions 26
5.2.5. Control macros 29
5.3. Additional functionality 34
5.3.1. Global variables 34
5.3.2. Bootloader 34
5.3.3. CONFIG bytes 35
5.3.4. Interrupts 36
5.3.5. Timed Responses 37
5.3.6. Status Display 38
5.3.7. Arduino functions 38
5.4. Reserving Flash memory 38
5.5. Writing a new Service 39
5.6. Writing a new Transport 40

6. Examples **41**
6.1. LED Blink 41
6.1.1. Module.h 41
6.1.2. Main.c 43
6.1.3. Compiling 46
6.1.4. Downloading 49
6.1.5. Testing 50
6.1.6. Debugging 50
6.2. LED Blink with variable blink rate 50
6.2.1. Changes to IDE project 51
6.2.2. Changes to module.h 51
6.2.3. Changes to main.c 52
6.2.4. Compiling and Downloading 53
6.2.5. Testing 53
6.3. LED Blink with enable/disable event 53
6.3.1. Changes to project 53
6.3.2. Changes to module.h 54
6.3.3. Changes to main.c 54
6.4. LED Blink with save of blink state 56
6.4.1. Changes to IDE project 56
6.4.2. Changes to module.h 56
6.4.3. Changes to main.c 56
6.4.4. Compiling and Downloading 57
6.4.5. Testing 57

1. Introduction

Within this document I attempt to describe the process of creating a PIC based module using the VLCBlib_PIC software library. The resulting module may be used on both VLCB and CBUS® networks.

I start by describing the overall process and then move onto a number of examples from a simple LED Blink module and then expand this to add more functionality.

It is recommended that the document “VLCB Technical Introduction” to obtain an understanding of CBUS and VLCB communications and events along with the VLCB Service model.

1.1. Location of library

This document refers to the VLCBlib_PIC library available from https://github.com/spikyian/VLCBlib_PIC

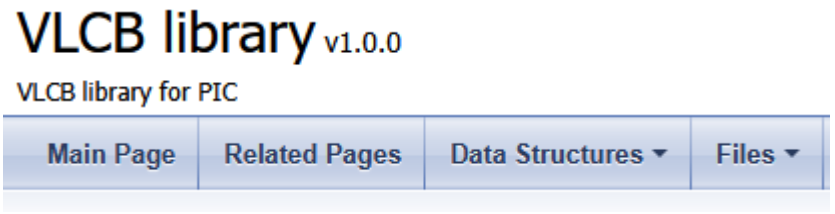
Online documentation of library functions and variables is available from https://spikyian.github.io/VLCBlib_PIC/

1.2. Releases and Versions

VLCBlib_PIC releases are made on the *main* branch of the VLCBlib_PIC repository. Other branches are used for development and testing of new features which are not yet released.

The release number for the library is available in the vlcb_version.h file. A user application may include this file and use conditional compilation (#if) to have release specific code.

The documentation indicates the version for which it applies with the release number string at the top of the page.



2. Setting up a Development Environment

2.1. Equipment Required

2.1.1. PC

A personal computer is required on which to develop your module code. This may use any of Windows, Linux or MacOS operating systems.

Sufficient hard disk space is required to load the development tools, library and your own developed code. Two USB ports (or one and an expander hub) will also be required.

2.1.2. PICkit

A PICkit is required to allow you to program and debug module code on a fresh PIC chip, without a bootloader. Although it may be possible to use a PIC chip from a MERG kit that contains a bootloader and use FCU to download your program it is far easier and quicker to download using the PICkit. To perform debugging operations such as variable inspection, single stepping and breakpoints the PICkit is required.

Care must be taken to ensure you have a compatible PICkit:

Processor	PICkit3	PICkit 4	PICkit5
PIC18F26K80	Supported	Supported	Supported
PIC18F27Q83	Not supported	Supported	Supported

2.1.3. PICkit Adapter cable

When programming and debugging the module's ICSP pins need to be connected to the PICkit. The MERG ICSP connector requires an adapter cable as described on the MERG site https://merg.org.uk/merg_wiki/doku.php?id=helpsystem:pickit2cbus

2.2. Software

2.2.1. MPLAB X IDE and MPLAB IPE

Download MPLAB X from the Microchip site. When this document was written the files could be downloaded from https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide but I cannot guarantee this link will work indefinitely, it may be necessary to use a web search engine to find the latest version. Note that to use the PICkit 5 (also known as ICD 5) MPLAB X v6.20 or later is required.

After the installation file has been downloaded, execute the file, accepting the default options to install MPLAB X IDE and IPE. It is the IDE (Integrated Development Environment) program which is used to develop programs.

There is also a lot of guidance on how to use MPLAB X on the Microchip site if additional information is required.

2.2.2. XC8 compiler

By default MPLAB X comes without any compilers installed and therefore a compiler suitable for the target device needs to be installed. It is necessary to install the Microchip XC8 compiler.

Download XC8 from the Microchip site. At the time of writing this is available from https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers/downloads-documentation#XC8

The October 2026 release of VLCBlib_PIC has been used with XC8 v3.0 and v3.1.

Once downloaded, run the installer program. The installer will obtain and register XC8 with MPLAB X IDE so that it is ready to use.

2.2.3. Git Tools

Since the library files are stored on the Github site it is recommended to download a set of Git tools. There is a wide choice of Git tools available, some with graphical user interfaces, others with command line interfaces. Any set of tools may be used.

For the Windows operating system I use https://git-scm.com/downloads and in the rest of this manual I will give instructions for use with Git Bash. This is comparable to a Linux shell so should also be able to be used with Linux and MacOS.

It is also possible to download the library files using the Github website as Zip files and unpacking locally but as it is easier to use the tools I will not cover this option any further.

2.3. Other downloads

Create a github folder and clone the following repositories. If using Git Bash you can use the following commands.

```
mkdir ~/github
```

2.3.1. VLCBlib_PIC

You will need a copy of the VLCBlib_PIC library.

```
cd ~/github
```

```
git clone https://github.com/spikyian/VLCBlib_PIC.git
```

Note the documentation for VLCBlib_PIC is in the folder `VLCBlib_PIC/doc/html`. Open the file `index.html` with a web browser to view the documentation.

2.3.2. VLCB-defs

You also need to obtain a copy of the VLCB-defs repository.

```
cd ~/github
git clone https://github.com/Versatile-LCB/VLCB-defs.git
```

2.3.3. CBUS_PIC_Bootloader

If you want to be able to use the FCU to download updated images to your module then you should also get a copy of the Bootloader repository.

```
cd ~/github
git clone https://github.com/MERG-DEV/CBUS_PIC_Bootloader.git
```

3. Module design

3.1. Software Structure

VLCBlib_PIC provides a framework for module software to ease the design and development process.

The module's application code needs to provide a `factoryReset()` function to initialise data in non volatile store, a `setup()` function in which any power-on initialisation should be performed and a `loop()` function which is called frequently.

The framework requires that the library and the application code work together with cooperative multitasking (https://en.wikipedia.org/wiki/Cooperative_multitasking). The `loop` function is called as often as possible by the framework so that the application can perform its functionality. The `loop` function must not block waiting for something to happen but must return from `loop` as soon as possible. If you require for work to be done later then the `loop` must set variables to indicate that work is to be done later and set up a timer variable which can be checked in `loop` to see whether the delay time has passed.

From a software structure point of view, it is recommended to divide the work needed to be done into logical threads so that there is a separate function for tasks such as checking for flashing LEDs, generating servo pulses, moving servo position. The `loop` function may look something like the following pseudo code.

```
TickValue servoPulseTimer;
TickValue servoMoveTimer;
TickValue flashLedTimer;

void loop(void) {
      if (tickTimeSince(servoPulseTimer) > 20*ONE_MILI_SECOND) {
            // call every 20ms
            doGenerateServoPulse();
      }
      if (tickTimeSince(servoMoveTimer) > 5*TEN_MILI_SECOND) {
            // call every 50ms
            doServoMove();
      }
      if (tickTimeSince(flashLedTimer) > ONE_SECOND) {
            // call every second
            doFlashLed();
      }
}
```

3.2. Functionality, NVs and EVs

Now that you have the equipment and software needed to write module software you now need to decide what your module is going to do. This might sound quite obvious but you

need to go into the detail of working out what configurable items will be needed by your module, whether these will be stored as *Node Variables* (NVs) or *Event Variables* (EVs).

3.3. Services

VLCBlib_PIC is structured around the concept of *Services* and you need to consider what services will be required by your module. You may need to write your own Service if an implementation isn't provided within VLCBlib_PIC.

Of course, your design may evolve as you refine both the design and implementation so don't worry too much about getting it right first time. We just need somewhere to start.

In some cases Services may have multiple implementations and as part of the design you will need to decide which implementation is needed to meet your requirements. In the case where none of the available implementations meet your needs then you may need to write your own Service implementation. The CAN service has different implementations depending upon the CAN peripheral available, see section 4.3.2.

3.4. Event Services

There are a number of Services associated with event processing:

- Event Teach service
- Event Producer service
- Event Consumer service
- Event CoE service

There are a number of different implementations for some of these services and a choice needs to be made on the implementation to be used.

3.4.1. Event Teach

Simple

The simple implementation uses a table of events containing rows with the event identifier and the event's EVs. The simple implementation is ideal for almost all circumstances.

To use the Simple event teach implementation include the event_teach.h header file and event_teach_simple.c files within your project.

The simple implementation uses less program space but more space is required for the event table than if the large implementation is used.

Large

The large implementation is useful when a large number of events or a large number of EVs are required. The large implementation stores a variable number EVs and allocates space for additional EVs as required.

To use the Large event teach implementation include the event_teach.h header file and event_teach_large.c files within your project.

The large implementation tries to reduce the amount of spaces required for the event table at the expense of more complex operation and associated increase in program size. Use the Large implementation only if you require events with a variable number of EVs, a maximum number of EVs or more than 20 and more than 130 events.

3.4.2. Event Producer

Basic

The basic event producer implementation is for the application itself to send an event using the sendMessage4() function:

```
sendMessage4(opc,
      producedEventNN.bytes.hi, producedEventNN.bytes.lo,
      producedEventEN.bytes.hi, producedEventEN.bytes.lo);
```

It is the application's responsibility to ensure the correct opcode, NN and EN are contained in the sent message.

Simple

The simple event producer implementation is an implementation of the event producer service. The service implementation handles the service, diagnostics and extended data. The service also provides a function to send an event from the event table:

```
void sendSimpleProducedEvent(uint8_t tableIndex, EventState state)
```

Happening

The Happening implementation of the event producer service uses the first EVs as a *Happening* number. The application can reference events to be produced using the Happening number.

To use the Happening event producer implementation include the event_producer.h and event_producer_happening.c files within your project.

3.4.3. Event Consumer

Simple

When an event is received the simple event consumer implementation checks that the event exists in the event table and then if it has been taught to the module it will call the application's `APP_processConsumedEvent()` function.

```
APP_processConsumedEvent(uint8_t tableIndex, Message * m);
```

To use the simple event consumer service include the event_consumer_simple.h and event_consumer_simple.c files within your project.

Actions

The Actions implementation of the event consumer service treats EVs as *Actions*. When an event is received the Actions are read from the EVs in the event table and put onto a queue for subsequent processing by the application.

The action queue implementation should be used when it could take an arbitrary length of time to perform the work required when consuming the event.

To use the actions event consumer service include the event_consumer_action.h and event_consumer_action.c files within your project.

3.5. Memory map

It is the responsibility of the module designer to define how memory is to be allocated in the module. The VLCBlib_PIC library provides a mechanism so that the type of non-volatile memory for NVs and EVs can be defined along with their address. If the compiler/linker were allowed to define the location of these assets then they could move every time the program was compiled but by fixing their location the program may be updated and still use the older version NV and EV values.

A typical memory map for a module with a larger number of events would be as the tables below.

Program Flash memory	PIC18F26K80	PIC18F27Q83
CBUS Bootloader	0000 ~ 07FF	00000 ~ 007FF
Vectors and Parameters	0800 ~ 084F	00800 ~ 00850
Program Space	0850 ~ Start of events -1	00850 ~ Start of events -1
Interrupt Vector Table		00900 ~ 009FF
Event table	Start of events ~ FFFF	Start of events ~ 1FFFF

Data EEPROM memory	PIC18F26K80	PIC18F27Q83
Data version	0000	0000
NV table	0001 ~ Number of NVs	0001 ~ Number of NVs
Available for program space		
AREQ storage		
Mode flags	03FA	03FA
Mode state	03FB	03FB
Node Number	03FC~03FD	03FC~03FD

CANID	03FE	03FE
Bootloader flag	03FF	03FF

RAM memory	PIC18F26K80	PIC18F27Q83
CAN Buffers		3BB0~3FFF

NVs require 1 byte per NV.

The current implementation of event storage is tuned for a large number of events with a large number of EVs. Each event takes 16 bytes of memory. In the future an alternate implementation may be provided which would be more suitable for smaller modules.

If there are a small number of NVs and EVs then it may be possible to fit all the configuration data in EEPROM space.

If NVs, Events or data which is not managed by the Linker is designed to be stored in Flash memory then it will be necessary to configure the project for the reserved memory addresses to be excluded from program space, see section 5.4.

3.6. Non Volatile Memory buffers

3.6.1. Flash memory

VLCBlib_PIC contains functionality to buffer changes to Flash memory and to write the changes to pages with page erases as necessary.

3.6.2. EEPROM

VLCBlib_PIC provides options for handling writing to EEPROM. Writes to the EEPROM memory may take a significant time period if the EEPROM needs to be erased before being written so in order to reduce the impact upon a user application the following strategies are supported:

Synchronous Direct writes

Whenever the user application requests an EEPROM write the write operation is performed immediately.

Asynchronous Buffered

When the user application requests a write to EEPROM the write data is written to a RAM buffer of the EEPROM data. The library implements a background task to scan the RAM buffer and writes any data needing to be written. The writes are done at a time when the user application is not running.

Asynchronous Queued

When the user application requests a write to EEPROM the write data and the address is saved to a RAM queue of changes to the EEPROM data. The library implements a background task to scan, extract and write any changes from the queue. The writes are done at a time when the user application is not running.

3.6.3. NV variables

Since NVs are likely to be read regularly by the application the VLCBlib_PIC library provides a NV cache so that NVs may be read directly from RAM rather than needing to be read from NVM. Writes to NVs are written to NVM and to the RAM cache.

4. Project setup in MPLAB X IDE

When you have sufficient confidence in your design to start development it is time to start to set up your project.

4.1. Create Project folder

If you are going to store your project in Github it is best to create the repository using the Github website. Press the green “New” button within the Repositories tab of your Github home page. Follow through the new repository process and then you can clone this empty repository to your local computer. For example if you call your first project “First” then give the Repository name as “First” as it is created in Github and after it is created clone it locally.

```
cd ~/github
git clone https://github.com/<Your name>/First.git
```

The URL to your new repository is also available from the green “<> Code” button on the repository page.

4.2. Create MPLAB X IDE Project

Start the MPLAB X IDE, as it starts it will check if there are any updates available. The first time you start the IDE there will undoubtedly be a few updates to be downloaded and it will probably be necessary to restart the IDE for these to take effect.

Select File → New Project menu item.

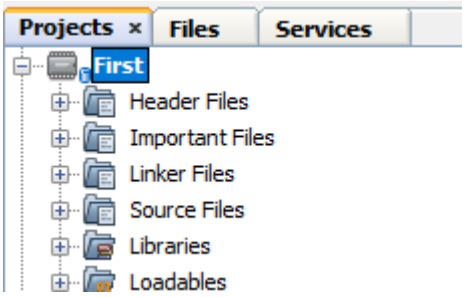
Select “Microchip Embedded” and “Application Project(s)” as the type of project. Press “Next >” to continue.

Select “Advanced 8-bit MPUs (PIC18)” as the Family, select your particular target device, currently this must be either PIC18F26K80 or PIC18F27Q83. Leave the Tool as “No Tool” for now, we will select the PICkit once we are ready to try the program. Press “Next >” to continue.

Select the XC8 compiler. Note if you have multiple compilers or multiple versions select the version required, this will normally be the latest version. Press “Next >” to continue.

Give your project a name. I recommend that you change the Project Location to be your project folder and tick the “Use project location as the project folder” checkbox. Untick the “Open MCC on Finish” checkbox. Press “Finish” to create the project.

The project will now be shown in the project hierarchy window.



4.3. Import Library files into project

The C source files and C header files from the repositories must be imported into the project.

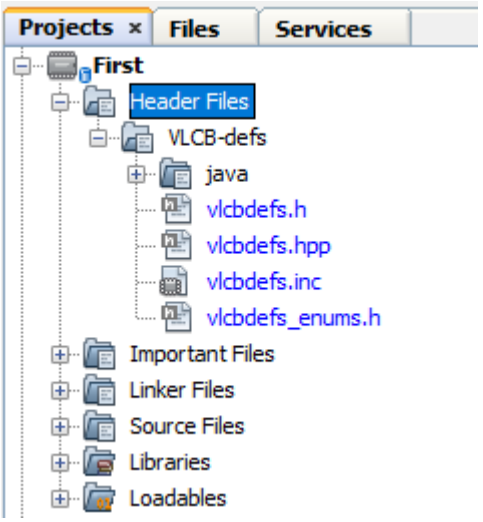
The IDE allows either a single file or an entire folder of files to be imported. Here we will choose to import the entire folder and we can remove the files we don't need later.

The source files ending in ".c" need to be imported into the "Source Files" section and the header files ending in ".h" need to be imported into the "Header Files" section of the project hierarchy.

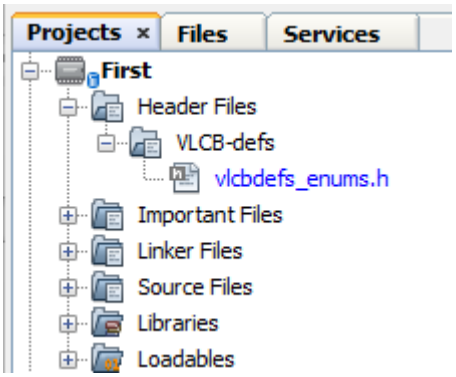
4.3.1. Import from VLCB-defs

Select "Header Files" in the project hierarchy and then right click and select "Add Existing Items from Folders..." from the pop-up menu. Select "Add Folder…" from the "Add Files" window.

Ensure "Header Files (.H .SUNWCCh .h .hpp .hxx .tcc .inc .INC)" is selected and then navigate to your github/VLCB-defs folder. Press "Select". Press "Add" to add the selected files. If you expand the Header Files group you can see all the files have been added.



We now have too many files imported and we should remove the files that we don't require. Right click on "java" and select "Remove From Project". Confirm the removal. Repeat the removal process for the "vlcbdefs.h", "vlcbdefs.hpp", "vlcbdefs.inc" items to leave just the "vlcbdefs_enums.h" item.



4.3.2. Import necessary files from VLCBlib_PIC

In a similar way to above import the header files from the VLCBlib_PIC.

Also import the file of "C Source Files (.c .i .m) from VLCBlib_PIC under the "Source Files" section.

Since most VLCB modules will require a CAN interface the minimum files required are:

- can.h
- devincs.h
- hardware.h
- messageQueue.h
- mns.h
- nvm.h
- statusDisplay.h
- statusLeds.h
- ticktime.h
- timedResponse.h
- vlcb.h

- can18_ecan.c — Change to match your particular CAN interface
- messageQueue.c
- mns.c
- nvm.c
- statusLeds2.c — Chose statusLeds1.c if your hardware has 1 status LED
- ticktime.c
- timedResponse.c
- vlcb.c

At this stage you can right click each file and remove the unnecessary files.

The particular CAN source code file selected will depend upon the CAN peripheral in the target PIC device.

The PIC18F26K80 device requires the can18_ecan.c source file. The PIC18F27Q83 device requires the can18_can_2.c source file.

If the module uses asynchronous EEPROM writes (ASYNC_EEPROM, section 5.2.4) also add asyncEEPROM.h and either asyncEEPROM_buffer.c or asyncEEPROM_queue.c.

It is expected that most target hardware will have 2 status LEDs, a green LED (possibly labelled SLiM) and a yellow LED (possibly labelled FLiM). For a module with 2 status LEDs use the statusLeds2.c source file. A module with only a single status LED should use statusLeds1.c.

4.3.3. Create project files

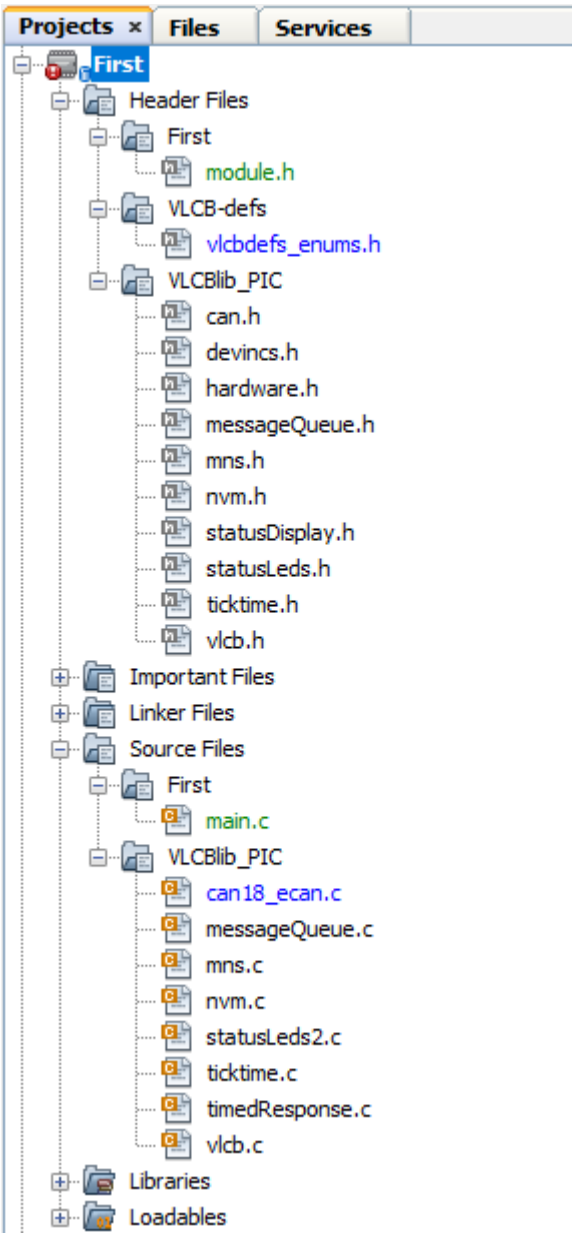
You need to create some files within your project folder.

```
cd ~/github/First
touch module.h
touch main.c
```

4.3.4. Import project files into project

Follow the import process in section 4.3.1 to import the project files into the IDE project.

The project hierarchy should now be as follows.



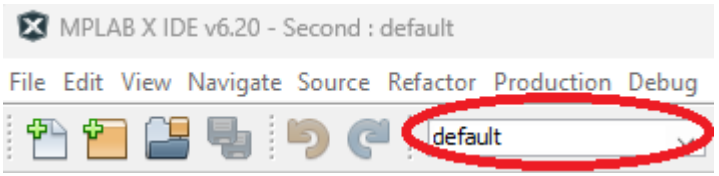
4.4. Project Properties

MPLAB X IDE needs to be told a bit more information to allow the project to build correctly. Right click on the name of the project and select “Properties…” from the pop-up menu.

Select “XC8 Compiler” from the Categories and in the “Include directories” element press the “…” button and browse to add “../../VLCB-defs” and “../../VLCBlib_PIC” and the project folder itself “../”

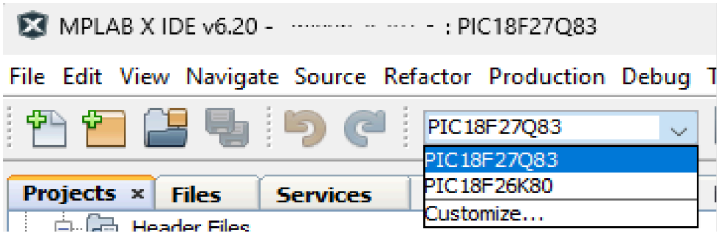
4.5. Updating Project for Different Processors

If there is a requirement to target different CPU types then it is recommended to utilise MPLAB X IDE *Configurations* function.



A Configuration holds details of the project such as target CPU type, compiler options, files included.

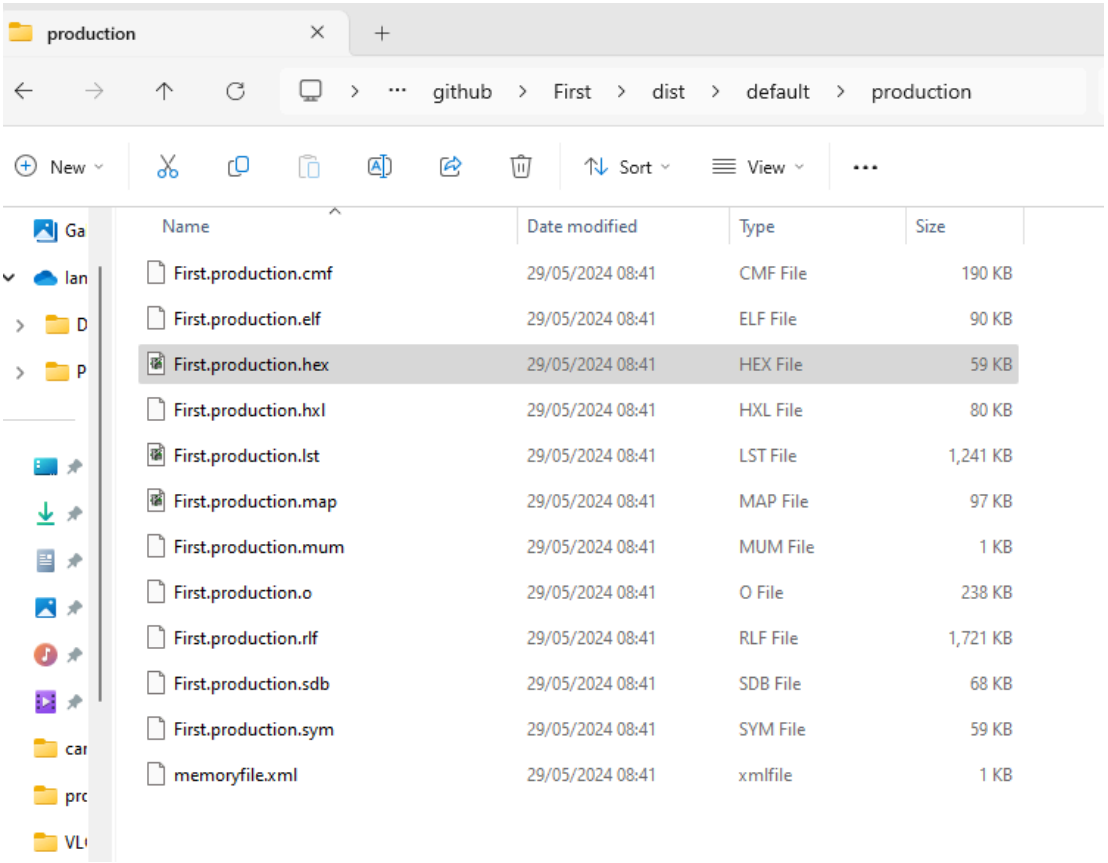
Configurations may be created, edited, deleted and renamed using the Customize option. Each Configuration has a section within the Project Properties.



When compiling the project the current configuration is used.

4.6. Compiler output

Whilst developing the compiler output is downloaded directly to the target board using the Download or Debug Main Project buttons. When it is time to distribute your file as a hex file then you can find the generated hex file under the IDE project directory. The IDE project directory was set in section 4.2 to be the same as the project folder created in section 4.1. If you navigate to your project folder you will see a *dist* folder which contains folders for each Configuration and then a folder called *production*. This folder contains all the generated files, including the project hex file.



One of the other generated files is the map file which can be useful to see how the linker has chosen to lay out the memory usage for your project.

5. Writing module software

5.1. Configuring the library in module.h

The file module.h contained within your project directory contains configuration required by the VLCBlib_PIC library. The details of what is required is documented within the docs directory of the VLCBlib_PIC although I provide a summary here.

5.1.1. MNS Configuration

Some configuration is required by all modules as it is needed by the VLCB framework or by the Minimum Node Service (MNS). The sample below can be used as a guide but care must be taken to ensure values are correct for your module.

```
#ifndef _MODULE_H_
#define _MODULE_H_

// comment out for CBUS
#define VLCB

#include "statusLeds.h"

//
// VLCB Service options first
//
// The application data version stored at NV#0
#define APP_NVM_VERSION 1

// Number of services in the services array
#define NUM_SERVICES 1

// Address of the IVT in Program Flash memory
#if defined(_18FXXQ83_FAMILY_)
#define IVT_BASE      0x900
#define IVT_BASE_U    0x00
#define IVT_BASE_H    0x09
#define IVT_BASE_L    0x00
#endif

//
// MNS service
//
// Processor clock speed
#define clkMHz                  64

// Configuration to define where Node number and state are stored
// 2 bytes for the module's node number
#define NN_ADDRESS              0x3FC
#define NN_NVM_TYPE             EEPROM_NVM_TYPE

// 1 byte for the mode
```

```
#define MODE_ADDRESS            0x3FB
#define MODE_NVM_TYPE           EEPROM_NVM_TYPE

// 1 byte for the application data version (APP_NVM_VERSION)
#define VERSION_ADDRESS 0x3FA
#define VERSION_NVM_TYPE EEPROM_NVM_TYPE

// 1 byte for the mode flags
#define MODE_FLAGS_ADDRESS      0x3F9
#define MODE_FLAGS_NVM_TYPE     EEPROM_NVM_TYPE

// Parameters
#define PARAM_MANU              MANU_MERG
#define PARAM_MODULE_ID         MTYP_MYMODULE
#define PARAM_MAJOR_VERSION     1
#define PARAM_MINOR_VERSION     'a'
#define PARAM_BUILD_VERSION     1
#define PARAM_NUM_NV            NV_NUM           // Number of NVs
                                                 // Specify 0 if NV service
                                                 // is not included
#define PARAM_NUM_EVENTS        NUM_EVENTS // Number of Events.
                                                 // Specify 0 if the event_teach
                                                 // service is not included
#define PARAM_NUM_EV_EVENT      1                // Number of EVs per event
// Module name - must be 7 characters. Pad with spaces if needed
#define NAME    "MYMOD  "

// LEDs and PB
// GREEN is 0 YELLOW is 1. Change the definitions below if the LEDs or PB
// are on non standard pins.
#if defined(_18F66K80_FAMILY_)
#define APP_setPortDirections(){ANCON0=ANCON1=0;
TRISBbits.TRISB6=TRISBbits.TRISB7=0,TRISAbits.TRISA2=1;}
#endif
#if defined(_18FXXQ83_FAMILY_)
#define APP_setPortDirections(){ANSELA=ANSELB=0;
TRISBbits.TRISB6=TRISBbits.TRISB7=0,TRISAbits.TRISA2=1;}
#endif
#define APP_writeLED1(state)    (LATBbits.LATB7=state)    // GREEN true is on
#define APP_writeLED2(state)    (LATBbits.LATB6=state)    // YELLOW true is on
#define APP_pbPressed()         (!(PORTAbits.RA2))        // where the push button
                                                          // is connected.
                                                          // True when pressed
#endif
```

5.1.2. Additional configuration for NV service

```
//
// NV service
//
#define NV_NUM                  1                 // Number of Nvs

#define NV_ADDRESS              0x200             // Address of NV storage
#define NV_NVM_TYPE             EEPROM_NVM_TYPE  // Type of NV storage
```

```
#define NV_CACHE                                // Define to cache NV values
                                                // in RAM
```

5.1.3. Additional configuration for CAN service

```
//
// CAN service
//
#define CANID_ADDRESS         0x3FE             // Address where CANID is stored
#define CANID_NVM_TYPE        EEPROM_NVM_TYPE   // Type of storage for CANID

#define CAN_INTERRUPT_PRIORITY 0                // all low priority

#define CAN_CLOCK_MHz         64                // CAN clock frequency. Normally
                                                // same as MNS'  clkMHz

// Number of software buffers
#if defined(_18F66K80_FAMILY_)
#define CAN_NUM_RXBUFFERS     32
#define CAN_NUM_TXBUFFERS     8
#endif
#if defined(_18FXXQ83_FAMILY_)
// On Q series normally use the hardware buffers but provide some software
// buffers for self consumed events
#define CAN_NUM_RXBUFFERS     8
#endif
```

5.1.4. Additional Configuration for Event Teach service

The current implementation of event_teach for modules require a large number of events and EVs requires the following configuration.

```
//
// EVENT TEACH SERVICE
//
//
#define EVENT_TABLE_WIDTH   10  // This is the width of the table - not the
                                // number of EVs per event as multiple rows in
                                // the table can be used to store an event
#define NUM_EVENTS          255  // The number of rows in the event table. The
                                // actual number of events may be less than this
                                // if any events use more the 1 row.
#define EVperEVT            20  // number of EVs per event
#define EV_FILL             0
#define NO_ACTION           0

#if defined(_18FXXQ83_FAMILY_)
 #define EVENT_TABLE_ADDRESS      0x1E000
#endif
#if defined(_18F66K80_FAMILY_)
 #ifdef __18F25K80
 #define EVENT_TABLE_ADDRESS      0x6F80      //Size=256 * 22 = 5632(0x1600)
```

```
 #endif
 #ifdef __18F26K80
  #ifdef CANXIO
   #define EVENT_TABLE_ADDRESS       0xE940   //Size=256 * 22 = 4096(0x1000)
 #else
   #define EVENT_TABLE_ADDRESS       0xEF80   //Size=256 * 22 = 5632(0x1600)
 #endif
#endif
#endif

#define EVENT_TABLE_NVM_TYPE   FLASH_NVM_TYPE  // Type of Event table storage

// Whether hash tables are used for quicker event lookup
#define EVENT_HASH_TABLE
// These are chosen so we don't use too much memory 32*20 = 640 bytes.
// Used to size the hash table used to lookup events in the events2actions table.
#define EVENT_HASH_LENGTH   32
#define EVENT_CHAIN_LENGTH  20
#define MAX_HAPPENING       10 // The number of different produced events
```

5.1.5. Additional Configuration for Event Producer service

```
//
// EVENT PRODUCER SERVICE
//
#define PRODUCED_EVENTS     // define when the event_producer service is present
#define HAPPENING_SIZE  1   // Happenings are 1 byte
```

5.1.6. Additional Configuration for Event Consumer service

```
//
// EVENT CONSUMER SERVICE
//
#define CONSUMED_EVENTS      // define when the event_consumer service is present
#define CONSUMER_EVS_AS_ACTIONS  // When defined the ACTION_QUEUE_SIZE must also
                                 // be defined. The event's EVs are treated as
                                 // Actions and added to the Action queue for
                                 // subsequent processing by the application.
                                 // If not defined then the function
                    // APP_processConsumedEvent(uint8_t tableIndex, Message * m)
                                 // must be defined by the application.
#define HANDLE_DATA_EVENTS       // Controls whether events with data are
                                 // treated the same as normal events
#define ACTION_QUEUE_SIZE      64  // MUST BE A POWER OF TWO
                                 // The size needs to be big enough to
                                 // store all the pending actions
#define ACTION_SIZE 1            // Actions are 1 byte
```

5.1.7. Additional configuration for BOOT service

```
//
// BOOT service
//
#define BOOT_FLAG_ADDRESS   0x3FF               // Address of the Boot flag
                                               // Must be set to 3FF in EEPROM
```

```
                                                // to be compatible with
                                                // CBUS PIC bootloaders
#define BOOT_FLAG_NVM_TYPE  EEPROM_NVM_TYPE// Type of Boot flag storage
#define BOOTLOADER_PRESENT                      // Always define when Bootloader
                                                // is used.
```

5.2. The main program source file

It is recommended that the module's setup() and loop() routines are held in the main.c file.

5.2.1. Header files

You must include the vlcb.h header file.

```
#include "vlcb.h"
```

Also include other header files for the services for example

```
#include "mns.h"
#include "nv.h"
#include "can.h"
```

Then include any other functionality from the library

```
#include "ticktime.h"
#include "nvm.h"
```

5.2.2. Services

Define the set of services your module is using. Remember that quantity in this list must match the NUM_SERVICES value in module.h

```
const Service * const services[] = {
    &canService,
    &mnsService,
    &nvService,
};
```

5.2.3. Transport

The majority of VLCB modules support a means of communication called a Transport.

The user application should include the relevant source file for the required Transport within the project and assign the global Transport variable called transport to the relevant Transport implementation. For example:

```
// use CAN as the module's transport
transport = &canTransport;
```

Some modules may support more than one communication Transports, typically these bridge VLCB messages between the Transports. The VLCB library looks after the first Transport, the user application should manage the Transports beyond the first.

VLCB supports different communications transport mechanisms. The most common transport type is CAN and VLCBlib_PIC provides different implementations of the CAN transport.

For CAN the library currently supports two mechanisms for including the hardware specific correct files.

Option 1. For PICs containing an ECAN peripheral such as the K series PICs then the project should include the can18_ecan.c source file. For PICs containing a CAN2.0 peripheral such as the Q series PICs then the project should include the can18_can_2.c source file. These files both implement the CAN service.

Option 2. Include the can_service.c file and then include a CAN hardware specific file. For a Q series PIC include both can_service.c and can_hw_q83.c source files. For a K series PIC include can_service. and can_hw_k80.c source files.

Functionally there is no difference between these options but Option 2 is preferred since Option 1 may be phased out in future releases of the library.

5.2.4. Callback functions

The VLCBlib_PIC library uses callback functions when it wants information from the application or when the library requires application specific functionality. Each Service or additional functionality may require additional callback functions to be provided by the application.

MNS callback functions

The following functions must always be provided by the application.

`void APP_factoryReset(void)`
The APP_factoryReset function is called when a fresh module is powered up for the first time. It is also called when the module receives the NNRSM opcode with the NN matching the module's.

The APP_factoryReset function should be used by the application to initialise any application specific data structures in non volatile memory.

`void APP_testMode(void)`
VLCB defines a sequence of pushing the module's push button at power up to allow the module's application to provide test facilities. The APP_testMode function is called whenever the push button sequence is entered to enter test mode.

`void APP_earlyInit(void)`
The APP_earlyInit() callback function is called as the very first operation by the VLCB library. This allows the user application to set up IO ports to an inactive state as soon as possible in order to reduce glitches on output pins. At this point in the setup process no

other configuration has taken place. Setup functionality which depends upon NVs or timers should be put into the setup() function instead.

`void setup(void)`
The setup function is called at module power up and allows the module to perform any power up processing such as initialising hardware to previous state.

`void loop(void)`
The module's loop function is called as frequently as possible to allow the module's application to perform its required processing.

`Processed APP_preProcessMessage(Message * m)`
The APP_preProcessMessage function is called by the VLCB framework when a message is received. The application may intercept the message before it is processed by the VLCB framework and services. If PROCESSED is returned then the message is not further processed by VLCB. If NOT_PROCESSED is returned then the message is passed to VLCB and the services.

Normally the application should just return NOT_PROCESSED.

`Processed APP_postProcessMessage(Message * m)`
Any messages that are not processed by the VLCB framework and services are passed to the APP_postProcessMessage function.

Normally the application should just return NOT_PROCESSED.

`APP_setPortDirections()`
This callback is normally provided as a #define macro in module.h. It is used toset up the IO ports used for the LEDs as outputs and the port used for the push button as an input. This macro should also ensure that the ports are configured to be digital instead of analogue.

`APP_writeLED1(state)`
A #define macro used by the MNS service to turn LED1 on or off.

`APP_writeLED2(state)`
A #define macro used by the MNS service to turn LED2 on or off.

`APP_pbPressed()`
A macro which should return true whilst the push button is pressed.

NOTE At this point you may wish to look at the first example module in section 6.1.

NV Service callback functions

The following function must be provided by the application if the NV service is used.

`uint8_t APP_nvDefault(uint8_t index)`
Called by the NV service during a factory reset to obtain the default value of an NV.

`void APP_nvValueChanged(uint8_t index, uint8_t value, uint8_t oldValue)`
The application APP_nvValueChanged function is called by the NV service whenever an NV has its value changed. This allows the application to perform additional processing upon the NV change.

`NvValidation APP_nvValidate(uint8_t index, uint8_t value)`
When a user requests that an NV value is changed the NV service calls the APP_nvValidate function to check whether the provided value is valid.

The application should return INVALID for specific NV values that cannot be accepted or VALID if the provided value is acceptable.

NOTE At this point check out the second example in section 6.2

Event Teach Service callback functions

The following callback functions must be provided if the event teach service is used.

`uint8_t APP_addEvent(uint16_t nodeNumber, uint16_t eventNumber, uint8_t evNum, uint8_t evVal)`
This function is called by the event teach service whenever an attempt is made to add an event by the EVLRN or EVLRNI opcodes. The application may implement a specialised event teaching or storage strategy.

Normally APP_addEvent is defined to be addEvent by a preprocessor macro.

Event consumer callback functions

If the event consumer service is used then the following callback functions must be provided.

`Processed APP_processConsumedEvent(uint8_t tableIndex, Message * m)`
If the preprocessor macro CONSUMER_EVS_AS_ACTIONS is not defined then the APP_processConsumedEvent callback function is called by VLCBlib_PIC whenever an event is received that has been taught to the module.

`uint8_t APP_isConsumedEvent(uint8_t tableIndex)`
The APP_isConsumedEvent callback function allows the library to query the user application whether the event at the table index is to be treated as a consumed event.

Event producer callback functions

`EventState APP_GetEventState(Happening h)`
The event producer service calls the application to determine the current state of an event in order to respond to a AREQ/ASRQ request. The application should respond with either EVENT_ON or EVENT_OFF.

`EventState APP_GetEventIndexState(uint8_t tableIndex)`
Allows the library to ask the application the current ON/OFF state of the specified event.

`uint8_t APP_isProducedEvent(uint8_t tableIndex)`
The APP_isProducedEvent callback function allows the library to query the user application whether the event at the table index is to be treated as a produced event.

NVM callback functions

`ValidTime APP_isSuitableTimeToWriteFlash(void)`
Writing flash is a time consuming process and the CPU is halted while the write is taking place. When the library is due to save data to Flash memory the APP_isSuitableTimeToWriteFlash application is called to check whether the application will need to do time critical work within the flash write timescale.

The application should return either GOOD_TIME or BAD_TIME.

The library will enter a tight loop calling APP_isSuitableTimeToWriteFlash waiting for GOOD_TIME. The application must make use of interrupts to update state and return GOOD_TIME once the time critical section is completed.

Interrupt callback functions

`void APP_lowIsr(void)`
`void APP_highIsr(void)`
These functions must be provided by the application when the interrupt vector table is not used.

See section 5.3.4 for information on handling interrupts.

5.2.5. Control macros

There are a set of macros which may be defined in module.h which allow fine control of some aspects of the VLCBlib_PIC library.

ACTION_SIZE

Used by event_consumer_action.c the ACTION_SIZE should be defined to be either 1 or 2 in order to specify how many EVs are used to specify an Action within the event table.

ASYNC_EEPROM

Used to specify the asynchronous EEPROM write methodology.

If set to BUFFER then EEPROM_BASE_ADDRESS and NUMBER_EEPROM must also be specified to define the range of EEPROM that is buffered in RAM.

If set to QUEUE then ASYNC_EEPROM_QUEUE_SIZE must also be specified to define the number of elements in the queue of EEPROM writes. If this value is set too small then EEPROM writes will be done synchronously.

Add asyncEEPROM.h to the project, and asyncEEPROM_buffer.c for BUFFER or asyncEEPROM_queue.c for QUEUE.

BOOTLOADER_PRESENT

The module should define this macro in module.h if the Boot service is included within the module and that a bootloader has been packed with the module code within the hex file.

CANID_ADDRESS

If a CAN service has been included then this should be defined as the one byte address at which the CANID will be stored. The type of the memory should also be defined in CANID_NVM_TYPE. Accepted types are EEPROM_NVM_TYPE or FLASH_NVM_TYPE.

CAN_ADDITIONAL_CANID_CHECKS

If CAN_ADDITIONAL_CANID_CHECKS is defined the additional code is included in the CAN service. The module performs additional checks to ensure that a CANID of zero is not used and that attempting to transmit using a CANID of zero will result in an additional automatic enumeration. Additional checks are also made to verify that an enumeration results in a valid CANID.

If a CANID enumeration fails to find a valid CANID the library will wait for ENUMERATION_RETRY number of ticks before retrying. If ENUMERATION_RETRY is not set in module.h then a value of FIVE_SECOND is used.

CAN_CLOCK_MHz

The normal processor speed based upon crystal frequency and PLL operation must be specified within module.h to allow the CAN bus speed to be correctly set at 125kb/s. It must be 16, 32 or 64.

CONSUMED_EVENTS

This macro must be defined in module.h if the module supports consumed events.

CONSUMER_EVS_AS_ACTIONS

Used by event_consumer_action.c to determine how EVs should be processed. If defined in module.h then EVs will be assumed to be a list of Actions and pushed onto an Action queue otherwise the callback function APP_processConsumedEvent() will be called.

EVENT_HASH_TABLE

If defined then a hash table will be used to speed up looking up of consumed events from the event table at the expense of more memory.

The define EVENT_PRODUCED_EVENT_HASH may also be specified to use a hash to speed looking up produced events.

The hash table size is specified by the defines EVENT_HASH_LENGTH and EVENT_CHAIN_LENGTH.

EVENT_TABLE_HEAL_ERASED

In the event of a power outage it is possible for an event table to contain partially written event information. If defined in module.h the EVENT_TABLE_HEAL_ERASED option includes a function which will check the event table and attempt to correct corrupted data.

FCU_COMPAT

If defined in module.h, the library will include additional functionality to support FCU.

HANDLE_DATA_EVENTS

If defined in module.h the library will treat events with data (e.g. ASON1, ACOF3) as normal events without the additional data bytes.

HAPPENING_SIZE

Used by event_consumer_action.c and event_producer_happening.c where an event's EVs are treated as a Happening followed by a number of Actions. This may be used to specify the size of a Happening in bytes and therefore at which EV the Actions begin. HAPPENING_SIZE may be set to either 1 or 2.

INDEX_EVENT

This must be defined if indexed events are being used by the module in which EVLRNI is used to specify an event by an index instead of by EVLRN using nn:en for event teaching.

NV_CACHE

Should be defined in module.h in order to cache NVs in RAM to provide quick access for reading NVs at the expense of more RAM memory.

PRODUCED_EVENTS

Must be defined in module.h if the module supports produced events.

SAFETY

When defined the library includes some additional run-time checks in event_teach to check that event indexes are within the bounds of the event table.

STRINGENT_TX_QUEUE_DRAIN

When the library needs to reset the CPU due to a NN_RST or prior to switching to the bootloader it is necessary to ensure that the CAN transmit queue is empty. If STRINGENT_TX_QUEUE_DRAIN is specified the the library will perform additional bus state checks to see if it is possible for the transmit queue to drain. This option is currently only available for the PIC Q83 family.

TX_DRAIN_TIMEOUT_MS

When the library needs to reset the CPU due to a NN_RST or prior to switching to the bootloader it is necessary to ensure that the CAN transmit queue is empty. The library will wait a maximum of TX_DRAIN_TIMEOUT_MS milliseconds for the transmit queue to drain. If undefined the timeout defaults to 500 ms.

VLCBLIB_VERSION

The library release version number is available in the VLCBLIB_VERSION_STRING. Individual parts of the version string are also available in VLCBLIB_VERSION_MAJOR, VLCBLIB_VERSION_MINOR and VLCBLIB_VERSION_PATCH. An integer representation of the version number is available as VLCBLIB_VERSION.

It is possible for a user application to check whether it is being compiled with a sufficiently recent library using:

```
#if VLCBLIB_VERSION < VLCBLIB_VERSION_NUMBER(2,0,0)
 #error "Require VLCBlib_PIV of release at least v2.0.0"
#endif
```

VLCB_VDD_GUARD

Defined in module.h as the HLVD voltage level code for the processor, for example 0x0B (about 4 V on both the PIC18F27Q83 and the PIC18F26K80). At power up, the library waits until Vdd has been above that level for VLCB_VDD_STABLE_MS milliseconds (default 50). It gives up after VLCB_VDD_STARTUP_MAX_MS milliseconds (default 2000) so that a module with a marginal supply still starts.

The user application must leave the HLVD module enabled. It may use the HLVD itself provided it keeps the same level.

VLCB_VDD_WRITE_GUARD

When defined in module.h, together with VLCB_VDD_GUARD, nvm.c checks that Vdd is above the VLCB_VDD_GUARD level before performing a NVM write.

If the Vdd has not become stable after VLCB_VDD_WRITE_WAIT_MS milliseconds then the write is refused; an EEPROM write returns an error and is counted as a memory error in the diagnostics, the page write is tried again at the next flush.

VLCB

If this is not defined in module.h then all VLCB extensions will be disabled and the module will only support CBUS functionality. Alternatively the individual VLCB enhancements may be enabled individually.

VLCB_CLRWDT()

If the user application uses watchdog timer (WDT) then it must define the VLCB_CLRWDT() macro to be called by the library in order to reset the watchdog.

VLCB_DEFAULT_ISR_HOOK & VLCB_DEFAULT_ISR_RESET

When using the interrupt vector table the user application may define a default interrupt handler if no other has been specified for the interrupt source. An unhandled interrupt would otherwise cause the CPU to go into an endless loop attempting to process the interrupt. This is a user application error.

When VLCB_DEFAULT_ISR_RESET is defined in module.h the module will reset if it encounters an unhandled interrupt instead of causing the CPU to go into an endless loop.

If VLCB_DEFAULT_ISR_HOOK is specified in module.h then the callback function APP_unhandledInterrupt() will be called before the reset.

VLCB_DIAG

If defined in module.h then the VLCB diagnostics functionality will be included within the library.

VLCB_GRSP

If defined in module.h then the VLCB GRSP response code will be included by the library.

VLCB_MODE

If defined in module.h then the VLCB MODE opcode will be included by the library.

VLCB_NVSETRD

If defined in module.h then the VLCB NVSETRD opcode will be included by the library.

VLCB_SERVICE

If defined in module.h then the VLCB RQSD, SD and ESD opcodes will be included by the library.

VLCB_LOOPS_PER_POLL & VLCB_MESSAGES_PER_POLL

The VLCBlib_PIC main loop typically checks whether there are any messages received from its transport interface, processing the message if one has been received and then calling poll() for each service and the user application loop() function. Some application types may need more priority given to servicing incoming transport messages and other applications may need more application loop priority.

The VLCB_LOOPS_PER_POLL and VLCB_MESSAGES_PER_POLL defines control the priority between these processes. If these values are not specified in module.h then they both default to 1.

clkMHz

The frequency of the clock (after PLL) should be defined within module.h as clkMHz as the number of megahertz of the clock frequency.

5.3. Additional functionality

5.3.1. Global variables

VLCBlib_PIC provides some global variables which may be used by the application.

`Word nn`
The current module node number.

`Mode_state mode_state`
The current state of the module; UNINITIALISED, SETUP or NORMAL.

`uint8_t mode_flags`
Toggle-able flags representing sub modes of the module; for example LEARN, HEARTBEAT, EVENT_ACK.

`const Service * const services[]`
The list of Services supported by the module.

`const Transport * transport;`
The communications transport object.

5.3.2. Bootloader

Add the CBUS_PIC_Bootloader project as a dependency to your project. Other bootloaders are available but the CBUS_PIC_Bootloader is covered here. To do this open the project properties by right clicking the project name in the Projects tab and select “Properties…” in the pop-up menu. In the “Loading” category press the “Add Loadable Project…” button and select the CBUS_PIC_Bootloader project, specifying the correct configuration to match your target processor.

The CBUS_PIC_Bootloader project has a number of configurations. These specify the PIC CPU type and also the PCB type. Specifically it is the CAN, LED and push button IO ports that are dependent upon the project configuration so that even if you are using an alternate PCB as long as these IO ports are the same as those in an existing configuration then you can use one of the existing configurations.

> Note If you are developing for a new PCB type then it will be necessary to update the hwsettings.h file with the new definitions similar to the following for the CANMIO family of PCBs:

```
#elif HARDWARE==HW_CANMIO
#define
SetPortDirections(){TRISBbits.TRISB6=TRISBbits.TRISB7=0,TRISAbits.
TRISA2=1;}
#define FLiM_SW         PORTAbits.RA2
#define LED1Y           LATBbits.LATB6  // Yellow LED
#define LED2G           LATBbits.LATB7  // Green LED
#define TRIS_LED1Y      TRISBbits.TRISB6
#define TRIS_LED2G      TRISBbits.TRISB7

#define LED_ON          1               // LEDs are active high
#define LED_OFF         0
#else
```

> A new project configuration can be created which specifies the new HARDWARE define for your PCB.

It is also necessary to tell the linker in your project to avoid the address space below address 0800 as that is reserved for the bootloader. In the “XC8 Linker” category of your project properties add “—CODEOFFSET=0x800” (note the double hyphen at the start and enter the text without the quotes) into the Additional options box.

Also ensure that the ROM range specified in the XC8 Linker Memory model starts at 800.

You will also need to add additional lines into module.h and include the Bootloader service in your project.

```
//
// BOOT service
//
#define BOOT_FLAG_ADDRESS   0x3FF
#define BOOT_FLAG_NVM_TYPE EEPROM_NVM_TYPE
#define BOOTLOADER_PRESENT
```

Adding the bootloader as a loadable project causes the bootloader project hex file to be merged into the current project using the hexmate component of MPLAB X IDE. After compiling the resulting output file from hexmate has a .unified.hex name ending.

The merge of the hex files will fail if there are any address contents conflicts i.e. if the hex files contain different contents at the same addresses then hexmate will provide an error and the unified.hex file will not be generated. If this happens then check that the current project is specified to use program memory space above 0x800 as described above.

5.3.3. CONFIG bytes

It is the responsibility of the bootloader, if present, to define the CONFIG bytes for the CPU. If the application wishes to define CONFIG bytes then they must not conflict with any bootloader. The recommended set of CONFIG bytes to be used by bootloaders and applications are available in the files:

- vlcb_config_k80.h
- vlcb_config_q83.h

If the module makes use of the CBUS_PIC_Bootloader then there is no need for the user application to specify any CONFIG byte settings.

The CONFIG bytes specified in the vlcb_config header files above wherever possible do not prevent the user application from turning on additional features and peripherals of the CPU. For example the user may enable a watchdog timer.

5.3.4. Interrupts

The application may enable hardware interrupts but the application must provide an interrupt service routine to handle the cause of the interrupt and clear the interrupt flag.

Warning - If the application fails to clear the interrupt flag the ISR will be called indefinitely leading to a module lock-up unless VLCB_DEFAULT_ISR_RESET is specified whereupon the CPU will reset instead.

Warning - The application is responsible for disabling interrupts using the function `di()` and re-enabling interrupts using the function `ei()` around time-critical sections of code or where concurrent access may corrupt data.

VLCBlib_PIC uses different interrupt service routine strategies depending upon processor type.

ISR strategy	High and Low interrupt priorities	Vectored interrupts using IVT
K series e.g. PIC18F26K80	✔	
Q series e.g. PIC18F27Q83		✔

High and Low interrupt priorities

When either a high priority or a low priority interrupt is received VLCBlib_PIC calls each service then it will call the APP_highIsr or APP_lowIsr callback function to service the interrupt.

If the application does not use any additional interrupts the application must provide the APP_highIsr and APP_lowIsr callback functions listed in section 5.2.3.7 although they may be empty functions.

If the application does enable interrupts for its own use then the application must provide the relevant content for one or both of the APP_highIsr and APP_lowIsr functions to check if the interrupt was caused by the interrupts it enabled and clear the interrupt flag.

Vectored interrupts

It is the responsibility of the Application to register its ISR into the IVT. For example if the application uses Timer1 and has enabled the Timer1 overflow interrupt then it must register a function as the ISR for Timer1:

```
void __interrupt(irq(TMR1), base(IVT_BASE)) TMR1_ISR(void) {
    if(PIR3bits.TMR1IF) {
        /* there was a timer overflow */
        PIR3bits.TMR1IF = 0;
        T1CONbits.TMR1ON = 0;
        ...
    }
}
```

On the PIC Q series interrupt priorities are also enabled. The library's own interrupts (CAN and the TMR0 tick) are low priority, so an application ISR declared without low_priority, as above, is high priority and can interrupt them. Set the source's IPR bit to match the declaration (for example IPR3bits.TMR1IP = 1 for the high priority TMR1_ISR below, or 0 for an ISR declared with low_priority): do not rely on the reset value, which differs between sources.

Clearing interrupt flag

It is the responsibility of the user application to enable only the interrupt sources for which an ISR has been included. The ISR must clear the interrupt flag otherwise the CPU will immediately go back into the ISR after leaving the ISR and the CPU will become unresponsive.

Unhandled interrupts

When using the interrupt vector table the user application may define a default interrupt handler if no other has been specified for the interrupt source. An *unhandled* interrupt would otherwise cause the CPU to go into an endless loop attempting to process the interrupt. This is a user application error.

When VLCB_DEFAULT_ISR_RESET is defined in module.h the module will reset if it encounters an unhandled interrupt instead of causing the CPU to go into an endless loop.

If VLCB_DEFAULT_ISR_HOOK is specified in module.h then the callback function APP_unhandledInterrupt() will also be called before the reset.

5.3.5. Timed Responses

When a large number of messages need to be sent there is both a possibility of exceeding the length of the transmit buffers and of swamping receiving nodes. It is therefore necessary to slow down the transmission of messages and also of not requesting that the messages are sent at the same time.

Instead a single request can be used to set up a TimedResponse using startTimedResponse which will generate and send the next message of a sequence on a regular basis. The startTimedResponse function takes a pointer to a callback function which is called each time a message is to be generated and sent. The callback function takes a uint8_t step parameter to indicate which message in the sequence to generate and send.

TimedResponse is used internally within the VLCBlib_PIC and care must be taken if it is used by the application to ensure requests do not clash with internal responses. It is recommended that the MNS service is specified for the application.

5.3.6. Status Display

Modules may have different ways of displaying state. Many modules use 2 LEDs, 1 green and 1 yellow. VLCB permits other methods such as a single LED or perhaps a LCD display.

The statusDisplay.h include file of VLCBlib_PIC includes an enum of statuses used by VLCB. VLCBlib_PIC also provides statusLeds1.c for displaying the status on 1 LED or statusLeds2.c for displaying the status on 2 LEDs. The project should include one of these source files.

If you wish to display status in another way then provide an alternate implementation for the functions:

```
void leds_powerUp(void)
void leds_poll(void)
void showStatus(StatusDisplay status)
```

5.3.7. Arduino functions

The arduino.h and ardunio.c files attempt to emulate the mechanism used on the Arduino system to access ports. There is a mapping from port/channel number to PIC PORTn bit m notation. The following functions are supported:

```
void pinMode(uint8_t channel, PinMode mode)
```

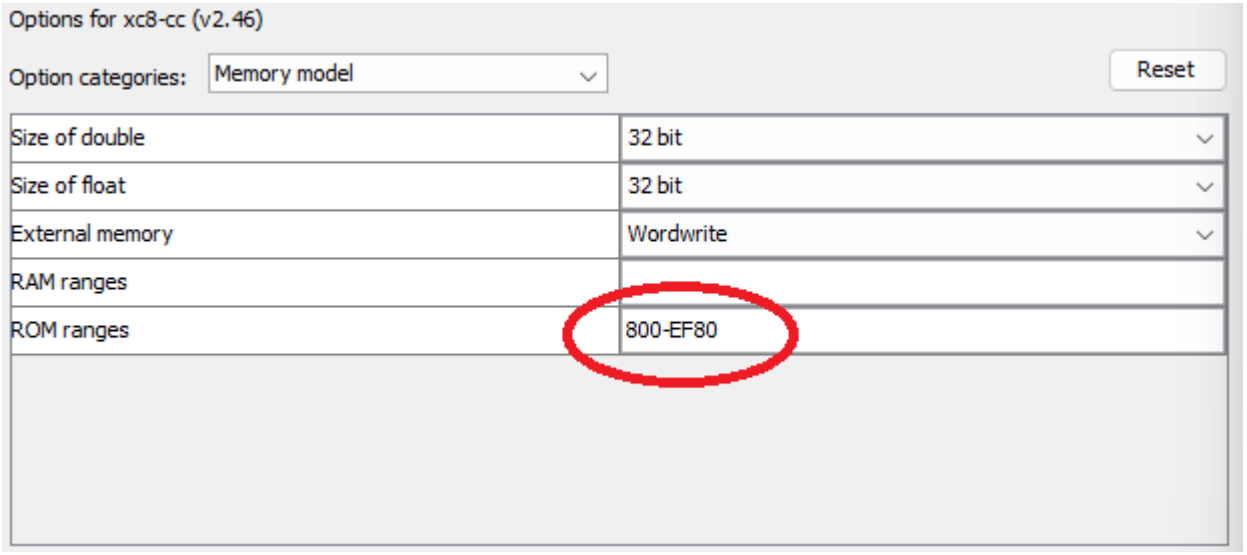
```
uint8_t digitalRead(uint8_t channel)
void digitalWrite(uint8_t channel, uint8_t value)
```

5.4. Reserving Flash memory

Whilst designing your module (see section 3.4) if you decided that Events or NVs will be stored in Flash memory then you need to tell the XC8 linker to not use this reserved space for the program.

Open the project properties and in the XC8 Global Options, XC8 Linker Category select “Memory model” from the Option categories pull-down and enter the addresses of flash to be used by the program within the ROM ranges entry.

When using the Bootloader this should start at 800 and any space reserved for Events and NV space at the top of memory.



When not using the bootloader the Flash space to be used by the program should start at 0.

5.5. Writing a new Service

Occasionally you may find that a service implementation by VLCBlib_PIC doesn’t quite meet your needs. In this case you are able to adapt or completely rewrite the service implementation.

In fact writing the main application is very similar to writing a service and the application can be thought of as a special service.

The service must provide a single instance of the Service type which can be put into the application’s service array.

```
typedef struct Service {
```

```
    uint8_t serviceNo;          ///< Identifies the type of service.
    uint8_t version;            ///< version of the service definition
    void (* factoryReset)(void);///< function call for a new module.
    void (* powerUp)(void);     ///< function called upon module power up.
    Processed (* processMessage)(Message * m);///< process any messages.
    void (* poll)(void);        ///< called regularly.
#if defined(_18F66K80_FAMILY_)
    void (* highIsr)(void);     ///< service specific high priority ISR.
    void (* lowIsr)(void);      ///< service specific high priority ISR.
#endif
#ifdef VLCB_SERVICE
    uint8_t (* getESDdata)(uint8_t id); ///< Return any service ESD bytes.
#endif
#ifdef VLCB_DIAG
    DiagnosticVal * (* getDiagnostic)(uint8_t index);///< pointer to function
                                                     //returning DiagnosticVal*.
#endif
} Service;
```

The new service may then provide any functionality required within these functions. Any unused functions should have a null function pointer within its Service instance.

5.6. Writing a new Transport

VLCB supports different communications transport mechanisms. The most common transport type is CAN and VLCBlib_PIC provides different implementations of the CAN transport.

It may be necessary to provide another CAN implementation for a different type of CAN engine or it may be necessary to provide a transport for a different communications stack such as USB, another type of CAN interface or BLE.

When a new Transport implementation is required then the new Transport should provide a singleton instance of the Transport object which can be used by the application to send and receive messages.

```
typedef struct Transport {
    SendResult (* sendMessage)(Message * m);    ///< function to send a message.
    MessageReceived (* receiveMessage)(Message * m); ///< check to see if a
                                                     // message is available and
                                                     // return it.
    TxDrainResult (* waitForTxQueueToDrain)(void); ///< wait until every
                                                     // queued message has
                                                     // been sent.
} Transport;
```

The waitForTxQueueToDrain function is called before the module resets, for example on NNRST or before entering the bootloader. It returns DRAIN_OK when the transmit queue is empty, or OVERALL_TIMEOUT, MESSAGE_TIMEOUT or BUS_OFF_ERROR.

For example the CAN Transport provides an instance of Transport with pointers to the canSendMessage, canReceiveMessage and canWaitForTxToDrain functions.

```
const Transport canTransport = {
    canSendMessage,
    canReceiveMessage,
    canWaitForTxToDrain
};
```

6. Examples

A number of examples of how to program a module using VLCBlib_PIC with increasing complexity.

Although this document takes you through the process of setting up these as projects in MPLAB X IDE they are also available ready to compile from https://github.com/spikyian/VLCBcookbookExamples

6.1. LED Blink

In typical fashion the first example will implement a blinking LED.

This is designed to work on a CANMIO or CANV*** board with the LED connected to channel 1.

Although it is possible to access channel 1 using TRISCbits.TRISC0 and LATCbits.LATC0 we shall instead use the arduino emulation file of VLCBlib_PIC so that pinMode() and digitalWrite() can be used to access channel 1. Ensure that arduino.h and arduino.c are added to the IDE project.

6.1.1. Module.h

Expand the project hierarchy so you can see the module.h file. Double click module.h which will then be shown as an empty file in the editor.

Copy and paste the following text into the module.h file.

```
#ifndef _MODULE_H_
#define _MODULE_H_

// comment out for CBUS
#define VLCB

#include "statusLeds.h"

//
// VLCB Service options first
//
// The data version stored at NV#0
#define APP_NVM_VERSION 1
#define NUM_SERVICES 2

// The hardware
#define CANMIO
#define NUM_IO 16
#if defined(_18FXXQ83_FAMILY_)
#define IVT_BASE      0x900
#define IVT_BASE_U    0x00
#define IVT_BASE_H    0x09
#define IVT_BASE_L    0x00
```

```
#endif

//
// CAN service
//
#define CANID_ADDRESS  0x3FE    // 1 byte
#define CANID_NVM_TYPE EEPROM_NVM_TYPE
#define CAN_INTERRUPT_PRIORITY 0    // all low priority
#define CAN_CLOCK_MHz   64
// Number of buffers
#if defined(_18F66K80_FAMILY_)
#define CAN_NUM_RXBUFFERS   32
#define CAN_NUM_TXBUFFERS   8
#endif
#if defined(_18FXXQ83_FAMILY_)
#define CAN_NUM_RXBUFFERS   8
#endif

//
// MNS service
//
// Processor clock speed
#define clkMHz      64

// 1 byte for the mode flags
#define MODE_FLAGS_ADDRESS    0x3F9
#define MODE_FLAGS_NVM_TYPE   EEPROM_NVM_TYPE
// 1 byte for the version number
#define VERSION_ADDRESS    0x3FA
#define VERSION_NVM_TYPE   EEPROM_NVM_TYPE
// 1 byte for the mode
#define MODE_ADDRESS    0x3FB
#define MODE_NVM_TYPE   EEPROM_NVM_TYPE
// 2 bytes for the module's node number
#define NN_ADDRESS  0x3FC
#define NN_NVM_TYPE EEPROM_NVM_TYPE

// Parameters
#define PARAM_MANU              MANU_MERG
#define PARAM_MODULE_ID         MTYP_CANDEV
#define PARAM_MAJOR_VERSION     1
#define PARAM_MINOR_VERSION     'a'
#define PARAM_BUILD_VERSION     1
#define PARAM_NUM_NV            0
#define PARAM_NUM_EVENTS        0
#define PARAM_NUM_EV_EVENT      0
// Module name - must be 7 characters
#define NAME    "BLINK  "
// LEDs and PB                                  // GREEN is 0 YELLOW is 1
#if defined(_18F66K80_FAMILY_)
#define APP_setPortDirections(){ANCON0=ANCON1=0;
TRISBbits.TRISB6=TRISBbits.TRISB7=0,TRISAbits.TRISA2=1;}
#endif
```

```
#if defined(_18FXXQ83_FAMILY_)
#define APP_setPortDirections(){ANSELA=ANSELB=0;
TRISBbits.TRISB6=TRISBbits.TRISB7=0,TRISAbits.TRISA2=1;}
#endif
#define APP_writeLED1(state)    (LATBbits.LATB7=state)    // GREEN true is on
#define APP_writeLED2(state)    (LATBbits.LATB6=state)    // YELLOW true is on
#define APP_pbPressed()         (!(PORTAbits.RA2))        // where the push button
is connected. True when pressed

#endif
```

6.1.2. Main.c

The listing of main.c for Blink is shown below. There is a lot of common code but the Blink specific code is highlighted in red. Copy the whole of the text below and paste into the empty main.c file.

```
/* The Blink example.
 *
 * @author Ian Hogg
 * @date May 2024
 *
 */
/**
 * @copyright Creative Commons Attribution-NonCommercial-ShareAlike 4.0
International License.
 */

#include <xc.h>
#include "module.h"
#include "vlcb.h"
// the services
#include "mns.h"
#include "can.h"
// others
#include "ticktime.h"
#include "timedResponse.h"
#include "devincs.h"
#include <stddef.h>
#include "statusLeds.h"
#include "nvm.h"
#include "timedResponse.h"
#include "arduino.h"

// The LED is to be connected to Channel 1 which is 0 in the code.
#define LED_CHANNEL 0

/*
 * File:   main.c
 * Author: Ian Hogg
 *
 * This is the example for Blink.
 */
```

```
static TickValue   blinkTime;
Boolean blinkState;

const Service * const services[] = {
    &canService,
    &mnsService,
};

/**
 * Called at first run to initialise all the non volatile memory.
 * Also called if the PB hold down special sequence at power up is done.
 * Also called as a result of a NNRSM request.
 */
void APP_factoryReset(void) {
}

/**
 * Called if the PB is held down during power up.
 * Normally would perform any test functionality to help a builder check the
hardware.
 */
void APP_testMode(void) {
}

/**
 * Called before anything else at power up, to put the module's own
 * outputs into a safe state as early as possible. Nothing to do for Blink.
 */
void APP_earlyInit(void) {
}

/**
 * Called upon power up.
 */
void setup(void) {
    // use CAN as the module's transport
    transport = &canTransport;

    /**
     * Set the timedResponse delay between messages.
     */
    setTimedResponseDelay(5);

    /*
     * Ensure LED channel is digital output.
     */
    pinMode(LED_CHANNEL, OUTPUT);

    /**
     * Initialise the blink timer
     */
    blinkTime.val = tickGet();
    blinkState = FALSE;
    // Initialise the LED output
```

```
    digitalWrite(LED_CHANNEL, blinkState);
}

void loop(void) {
    // Written out in full to make it easier to understand
    // Check if it is time to change the LED state
    if (tickTimeSince(blinkTime) > ONE_SECOND) {
        if (blinkState) {
            // If it was TRUE then now make it FALSE
            blinkState = FALSE;
        } else {
            // If it was FALSE now make it TRUE
            blinkState = TRUE;
        }
        // Update the channel 1 output
        digitalWrite(LED_CHANNEL, blinkState);
        // Update the timer so we called back at the right time
        blinkTime.val = tickGet();
    }
}

// Application functions required by VLCB library
/**
 * Check to see if now is a good time to start a flash write.
 * Always GOOD
 */
ValidTime APP_isSuitableTimeToWriteFlash(void){
    return GOOD_TIME;
}

/**
 * This application doesn't need to process any messages in a special way.
 */
Processed APP_preProcessMessage(Message * m) {
    return NOT_PROCESSED;
}

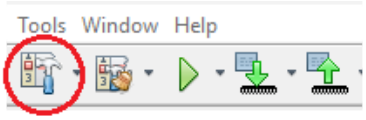
/**
 * This application doesn't need to process any messages in a special way.
 */
Processed APP_postProcessMessage(Message * m) {
    return NOT_PROCESSED;
}

#if defined(_18F66K80_FAMILY_)
// APP Interrupt service routines
void APP_lowIsr(void) {
}

// Interrupt service routines
void APP_highIsr(void) {
}
#endif
```

6.1.3. Compiling

The Build button on the IDE toolbar is used to compile the project. You have the choice to build for debugging but lets just check for any compile problems by using “Build Main Project”. Next to the build button there is a “Clean and Build” button which is sometimes needed if the system gets confused about the compile state of files.



The results of the compilation attempt will be shown at the bottom of the window. If any errors are detected then an error is shown with a hint to what and where the error might be. Sometimes the error messages are a bit cryptic but study the code for errors or search the web for the meaning of the error.

```
make -f nbproject/Makefile-default.mk SUBPROJECTS= .build-conf
make  -f nbproject/Makefile-default.mk dist/default/production/Blink.production.hex
make[2]: *** [nbproject/Makefile-default.mk:216: build/default/production/main.p1] Error 1
make[1]: *** [nbproject/Makefile-default.mk:85: .build-conf] Error 2
make: *** [nbproject/Makefile-impl.mk:39: .build-impl] Error 2
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/main.p1 main.c
main.c:178:25: error: expected ';' after return statement
    return NOT_PROCESSED
                        ^
                        ;
1 error generated.
(908) exit status = 1
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'

BUILD FAILED (exit value 2, total time: 618ms)
```

Once all errors have been fixed the compile process will still generate some warnings about duplicated functions and functions that are not used. This is normal at this stage.

```
make -f nbproject/Makefile-default.mk SUBPROJECTS= .build-conf
make  -f nbproject/Makefile-default.mk dist/default/production/Blink.production.hex
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/timedResponse.p1 ../../VLCBlib_PIC/timedResponse.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
```

```
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/arduino.p1 ../../VLCBlib_PIC/arduino.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/main.p1 main.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/can18_ecan.p1 ../../VLCBlib_PIC/can18_ecan.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/messageQueue.p1 ../../VLCBlib_PIC/messageQueue.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/mns.p1 ../../VLCBlib_PIC/mns.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/nvm.p1 ../../VLCBlib_PIC/nvm.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/statusLeds2.p1 ../../VLCBlib_PIC/statusLeds2.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/ticktime.p1 ../../VLCBlib_PIC/ticktime.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80 -c   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
```

```
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a -DXPRJ_default=default
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits   -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto     -o
build/default/production/_ext/1954642981/vlcb.p1 ../../VLCBlib_PIC/vlcb.c
make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
make[2]: Entering directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'
"C:\Program Files\Microchip\xc8\v2.46\bin\xc8-cc.exe"  -mcpu=18F26K80
-Wl,-Map=dist/default/production/Blink.production.map  -DXPRJ_default=default
-Wl,--defsym=__MPLAB_BUILD=1   -mdfp="C:/Program
Files/Microchip/MPLABX/v6.20/packs/Microchip/PIC18F-K_DFP/1.13.292/xc8"  -fno-short-double
-fno-short-float -memi=wordwrite -O0 -fasmfile -maddrqual=ignore -xassembler-with-cpp
-I"../../VLCB-defs" -I"../../VLCBlib_PIC" -I"." -mwarn=-3 -Wa,-a
-msummary=-psect,-class,+mem,-hex,-file  -ginhx32 -Wl,--data-init -mno-keep-startup -mno-download
-mno-default-config-bits -std=c99 -gdwarf-3 -mstack=compiled:auto:auto:auto
-Wl,--memorysummary,dist/default/production/memoryfile.xml -o
dist/default/production/Blink.production.elf  build/default/production/main.p1
build/default/production/_ext/1954642981/arduino.p1
build/default/production/_ext/1954642981/can18_ecan.p1
build/default/production/_ext/1954642981/messageQueue.p1
build/default/production/_ext/1954642981/mns.p1 build/default/production/_ext/1954642981/nvm.p1
build/default/production/_ext/1954642981/statusLeds2.p1
build/default/production/_ext/1954642981/ticktime.p1
build/default/production/_ext/1954642981/timedResponse.p1
build/default/production/_ext/1954642981/vlcb.p1
../../VLCBlib_PIC/can18_ecan.c:693:: advisory: (1510) non-reentrant function "_getBufferPointer"
appears in multiple call graphs and has been duplicated by the compiler
../../VLCBlib_PIC/messageQueue.c:82:: advisory: (1510) non-reentrant function
"_getNextWriteMessage" appears in multiple call graphs and has been duplicated by the compiler
../../VLCBlib_PIC/messageQueue.c:98:: advisory: (1510) non-reentrant function "_pop" appears in
multiple call graphs and has been duplicated by the compiler
../../VLCBlib_PIC/mns.c:641:: advisory: (1510) non-reentrant function "_updateModuleErrorStatus"
appears in multiple call graphs and has been duplicated by the compiler
../../VLCBlib_PIC/ticktime.c:158:: advisory: (1510) non-reentrant function "_tickGet" appears in
multiple call graphs and has been duplicated by the compiler
C:\Program Files\Microchip\xc8\v2.46\pic\sources\c99\common\Umul16.c:15:: advisory: (1510)
non-reentrant function "___wmul" appears in multiple call graphs and has been duplicated by the
compiler
../../VLCBlib_PIC/arduino.c:244:: warning: (520) function "_digitalRead" is never called
../../VLCBlib_PIC/messageQueue.c:115:: warning: (520) function "_peek" is never called
../../VLCBlib_PIC/timedResponse.c:82:: warning: (520) function "_timedResponseInProgress" is
never called
../../VLCBlib_PIC/vlcb.c:698:: warning: (520) function "_findService" is never called
../../VLCBlib_PIC/vlcb.c:1048:: warning: (520) function "_checkLen" is never called
../../VLCBlib_PIC/vlcb.c:1070:: warning: (520) function "_isEvent" is never called
../../VLCBlib_PIC/vlcb.c:1078:: warning: (520) function "_sendMessage0" is never called
../../VLCBlib_PIC/vlcb.c:1087:: warning: (520) function "_sendMessage1" is never called
::: advisory: (2087) memory for the main software stack has been reserved in response to the
detection of functions built for reentrancy
::: advisory: (2087) memory for the high priority software stack has been reserved in response to
the detection of functions built for reentrancy
::: advisory: (2087) memory for the low priority software stack has been reserved in response to
the detection of functions built for reentrancy

Memory Summary:
    Program space        used  5E50h ( 24144) of 10000h bytes   ( 36.8%)
    Data space           used   2F3h (   755) of   E41h bytes   ( 20.7%)
    Configuration bits   used     7h (     7) of     7h words   (100.0%)
    EEPROM space         used   3FBh (  1019) of   400h bytes   ( 99.5%)
    ID Location space    used     0h (     0) of     8h bytes   (  0.0%)
    Data stack space     used   B2Ch (  2860) of   B2Ch bytes   (100.0%)

make[2]: Leaving directory 'C:/Users/ianwh/github/VLCBcookbookExamples/Blink'

BUILD SUCCESSFUL (total time: 2s)
Loading code from
C:/Users/ianwh/github/VLCBcookbookExamples/Blink/dist/default/production/Blink.production.hex...
Program loaded with pack,PIC18F-K_DFP,1.13.292,Microchip
Loading completed
```

The important but is the “BUILD SUCCESSFUL” near the end of the output. This will produce a file called ~/github/First/dist/default/production/First.production.hex

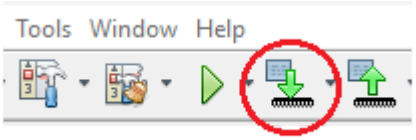
6.1.4. Downloading

Once you have a successful build you are able to download your program to the PIC.

Connect the PICkit to your computer’s USB port and connect the PICkit to your test board’s ICSP port. If this is a MERG board it is likely that you will need the adapter cable as described on the MERG wiki https://merg.org.uk/merg_wiki/doku.php?id=helpsystem:PICkit2cbus. Note that the adapter cable should be kept as short as possible to eliminate errors.

Also refer to the table in section 2.1.2 for which version of the PICkit can be used for the different processors.

Once connected correctly pressing the download button will go through the process to download to the PIC.



The first time this is done the IDE will ask you to select the Tool to be used for downloading. If the PICkit is not shown in the list of available tools then it may be necessary to restart the IDE to get it to recognise that the newly plugged in PICkit is available.

It is recommended that during programming that the test board is powered and that the option to supply power from the PICkit is not selected.

```
**************************************************



Connecting to MPLAB PICkit 5

Currently loaded versions:
Application version...........02.01.60
Boot version..................00.00.10
PCB version...................2
Script version................00.07.19
Script build number...........343401fe73
Tool pack version ............2.4.366
Target voltage detected
Target device PIC18F26K80 found.
```

```
Device Revision Id  = 0x6
Device Id = 0x6120

Calculating memory ranges for operation...

Erasing...

The following memory area(s) will be programmed:
program memory: start address = 0x0, end address = 0x3f
program memory: start address = 0x840, end address = 0x5a3f
program memory: start address = 0xfec0, end address = 0xffff
configuration memory

Programming/Verify complete
```

Once programming is complete the PIC will be released from reset and start running the Blink program. Ensure you have a LED connected to channel 1 to see the fruits of your labour.

6.1.5. Testing

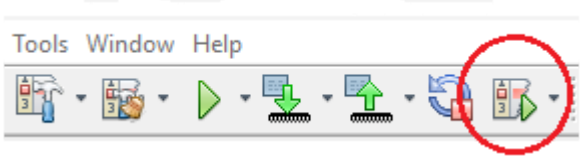
Since we included the MNS and CAN service in the example we can also connect up the test module into a CBUS network and register it with FCU.

When the Properties are read you should see the module described as CANBLINK version 1a1 with no NVs and no events.

6.1.6. Debugging

At some point in your module development (probably early on) you will get to the situation in which the module doesn't do what you expect. Inspecting the code doesn't reveal a cause of the problem and even trying to consider all possible causes still doesn't provide an answer of why the module is misbehaving. At this point you will need to inspect the variables and state of the running module to gather more information about the problem.

With the PICkit plugged into the ISCP port of the module you can select the "Debug Main Project" button.



This will recompile the project for debugging and download it to the target module.

It is possible to pause the program, set breakpoints, single step through the code and inspect variables and registers. Use these functions whilst recreating the problem to determine why the code is not following its intended path.

6.2. LED Blink with variable blink rate

Let's now extend the first blink example by adding a NV to control the blink rate. So we'll need to add the NV service, include the NV service configuration within module.h and update the main loop to use the NV value. We'll also need to provide a default value for the NV.

We shall encode the NV value as a number of 100 ms to wait and our default value shall be 10 i.e. one second. A value of 0 for the NV will be treated as invalid.

6.2.1. Changes to IDE project

The file nv.h needs to be added to the VLCBlib_PIC header files included in the project. To do this right click on the VLCBlib_PIC section in the Header Files section of the Project hierarchy and select "Add Existing Item…" then select the nv.h file.

Similarly the file nv.c needs to be added to the VLCBlib_PIC Source files included in the project.

6.2.2. Changes to module.h

Since we will be including a new Service within the application the number of services must be increased. Change the line

```
#define NUM_SERVICES 2
```

to

```
#define NUM_SERVICES 3
```

There are a couple of configurations needed by the NV service so add the following lines towards the bottom of module.h, just before the final #endif

```
//
// NV Service
//
// Number of NVs
#define NV_NUM  1
#define NV_ADDRESS  0x200
#define NV_NVM_TYPE EEPROM_NVM_TYPE
#define NV_CACHE
```

We also will define a helpful identifier for the single NV we will have. This doesn't need to be in the module.h file, we could put it into a new application specific file but for simplicity in this example we'll add it to module.h so also add the following lines towards the bottom of the file.

```
// The NV indexes
#define NV_LOOP_TIMER 1
```

So that FCU knows the module has an NV, change the parameter line

```
#define PARAM_NUM_NV 0
```

to

```
#define PARAM_NUM_NV NV_NUM
```

6.2.3. Changes to main.c

Add the nv include file to main.c

```
// the services

#include "mns.h"
#include "can.h"
#include "nv.h"
```

Add the NV service to the list of services:

```
const Service * const services[] = {
    &canService,
    &mnsService,
    &nvService,
};
```

The `loop()` function also needs to be updated to use the timer value from the new NV. We'll also make the loop more compact.

```
void loop(void) {
    // Written out in full to make it easier to understand
    // Check if it is time to change the LED state
    if (tickTimeSince(blinkTime) > HUNDRED_MILI_SECOND * getNV(NV_LOOP_TIMER)) {
        // Toggle blinkState variable
        blinkState = ! blinkState;
        // Update the channel output
        digitalWrite(LED_CHANNEL, blinkState);
        // Update the timer so we called back at the right time
        blinkTime.val = tickGet();
    }
}
```

When the NV service is used by the application then the application must provide an additional three callback functions. The following functions need to be added to the bottom of main.c

```
/********************* NV Service *********************************/
/*
 * NV default values.
 */
uint8_t APP_nvDefault(uint8_t index) {
    switch (index) {
        case NV_LOOP_TIMER:
            return 10;
        default:
            return 0;
```

```
    }
}

/*
 * Called when a NV value is changed.
 * No need to do anything for the Blink 2 example
 */
void APP_nvValueChanged(uint8_t index, uint8_t newValue, uint8_t oldValue) {

}

/*
 * Validate that the provided new value is acceptable.
 */
NvValidation APP_nvValidate(uint8_t index, uint8_t value) {
    switch (index) {
        case NV_LOOP_TIMER:
            if (value == 0) {
                return INVALID; // A loop timer value of 0 is invalid
            } else {
                return VALID;
            }
        default:
            return 0;
    }
}
```

6.2.4. Compiling and Downloading

Compile and download the updated application in the same way as the first example.

6.2.5. Testing

In the same way as the previous example the LED on channel 1 should flash at 0.5Hz after downloading. The module can then be registered with FCU and the NVs can be read. The single NV should have a value of 10. If the NV is changed and the NV written back to the module then the LED flash rate should change.

6.3. LED Blink with enable/disable event

For this example we shall extend the example of section 6.2 to add an event consumer service which can be used to turn the flashing on or off. We shall also add an event Teach service to allow our chosen event to be taught to the module.

In this example we have just 1 event and so the simple teach implementation is more than adequate. The functionality to perform the flash on/off operation is very quick to execute so therefore there is no need to use the action event consumer implementation and the simple event consumer implementation is chosen.

6.3.1. Changes to project

Add the following files to the VLCBlib_PIC section of the Header Files in the project hierarchy.

- event_consumer_simple.h
- event_teach.h

Add the following files to the VLCBlib_PIC section of the SourceFiles in the project hierarchy.

- event_consumer_simple.c
- event_teach_simple.c

6.3.2. Changes to module.h

Edit module.h to make the following changes.

The number of services definition needs to be changed from 3 to 5 for the added two services.

```
#define NUM_SERVICES 5
```

The configuration for the Simple event teach service and the simple event consumer service should be added.

```
//
// Event Teach
//
#define NUM_EVENTS       1  // The number of rows in the event table. The
                               // actual number of events may be less than this
                               // if any events use more the 1 row.
#define EV_FILL          0
#define EVENT_TABLE_ADDRESS    0x100
#define EVENT_TABLE_NVM_TYPE   EEPROM_NVM_TYPE

//
// Event Consumer service
//
#define APP_addEvent     addEvent
```

The following changes to the MNS parameter block should be made. Note that this uses the NUM_EVENTS definition from the Event Teach section so the parameter section should be moved to the bottom of the file, just before the #endif

```
#define PARAM_NUM_EVENTS       NUM_EVENTS

#define PARAM_NUM_EV_EVENT     1
```

Note that although we define 1 EV per event the example doesn't make use of the EV but the Teach opcode and Teach service require a minimum of 1 EV.

6.3.3. Changes to main.c

Update the main.c file to add the necessary include statements.

```
#include <xc.h>
#include "module.h"
#include "vlcb.h"
// the services
#include "mns.h"
#include "can.h"
#include "nv.h"
#include "event_teach.h"
#include "event_consumer_simple.h"
```

We also need to include a declaration for the blinkOn variable and also to include the additional 2 services. Note that the eventConsumerService is added as the first element of the array of Services as that is slightly more efficient.

```
static TickValue   blinkTime;
Boolean blinkState;
Boolean blinkOn;

const Service * const services[] = {
      &eventConsumerService,
      &canService,
      &mnsService,
      &nvService,
      &eventTeachService,
};
```

Add the initialisation of the blinkOn variable into the setup() function.

```
      blinkTime.val = tickGet();
      blinkState = FALSE;
      blinkOn = TRUE;
```

The loop() function is updated to make use of the blinkOn variable

```
void loop(void) {
  // Written out in full to make it easier to understand
  // Check if it is time to change the LED state
  if (blinkOn) {
    if (tickTimeSince(blinkTime) > HUNDRED_MILI_SECOND * getNV(NV_LOOP_TIMER)) {
      // Toggle blinkState variable
      blinkState = ! blinkState;
      // Update the channel output
      digitalWrite(LED_CHANNEL, blinkState);
      // Update the timer so we called back at the right time
      blinkTime.val = tickGet();
    }
  } else {
    digitalWrite(LED_CHANNEL, FALSE);   // Force OFF
  }
}
```

The function for handling the events is also added. Note that we use a bit mask on the event message opcode (m->opc) to determine whether the flash should be on (ACON opcode) or off (ACOF opcode).

```
/************************** Consumer Simple Service ************************/
Processed APP_processConsumedEvent(uint8_t tableIndex, Message * m) {
```

```
        // Check to see what to do with the event
        // In this minimal example we have the event stored and we use it to
        // enable or disable the blinking LED.
        blinkOn = !(m->opc & EVENT_ON_MASK);
        return PROCESSED;
}
```

The simple event consumer also asks the application whether an event in the event table is one it consumes. In this example every event taught to the module is consumed, so add:

```
/*
 * Every event taught to this module is a consumed event.
 */
uint8_t APP_isConsumedEvent(uint8_t tableIndex) {
    return 1;
}
```

6.4. LED Blink with save of blink state

In the example of section 6.3 the module always starts blinking when it is powered up, even if an OFF event had turned the blinking off. In this example we save the on/off state in EEPROM so that the module remembers it across a power cycle.

VLCBlib_PIC provides readNVM() and writeNVM() to read and write a byte of EEPROM or Flash. We shall use one byte of EEPROM that is not used by the library or by any of the services. The completed example is BlinkSave in the examples repository.

6.4.1. Changes to IDE project

No new library files are needed. nvm.h and nvm.c are already in the project.

6.4.2. Changes to module.h

Add the address of the saved state towards the bottom of module.h, just before the final #endif.

```
//
// Application data
//
// 1 byte to save whether blinking is on, so it survives a power cycle
#define BLINK_ON_ADDRESS 0x3F8
#define BLINK_ON_NVM_TYPE EEPROM_NVM_TYPE
```

The address must not be used by anything else in the module. In this example the event table is at 0x100, the NVs are at 0x200 and the library uses 0x3F9 to 0x3FF.

6.4.3. Changes to main.c

Set the saved state in APP_factoryReset() so that a new module starts with blinking turned on.

```
void APP_factoryReset(void) {
    // A new module starts with blinking turned on
    writeNVM(BLINK_ON_NVM_TYPE, BLINK_ON_ADDRESS, TRUE);
}
```

In setup() replace the line blinkOn = TRUE; with:

```
// Restore whether blinking was on or off when the module was last powered.
// readNVM returns a negative value if the read failed: blink in that case.
blinkOn = (readNVM(BLINK_ON_NVM_TYPE, BLINK_ON_ADDRESS) == FALSE) ? FALSE : TRUE;
```

Then update APP_processConsumedEvent() to save the new state. It only writes when the state changes: EEPROM can only be written a limited number of times, and the same event may be sent many times.

```
Processed APP_processConsumedEvent(uint8_t tableIndex, Message * m) {
    // ON events have even opcodes
    Boolean newOn = (m->opc & EVENT_ON_MASK) ? FALSE : TRUE;
    if (newOn != blinkOn) {
        blinkOn = newOn;
        // Save only on a change: EEPROM has a limited number of write cycles
        writeNVM(BLINK_ON_NVM_TYPE, BLINK_ON_ADDRESS, blinkOn);
    }
    return PROCESSED;
}
```

6.4.4. Compiling and Downloading

Compile and download the updated application in the same way as the first example.

6.4.5. Testing

Teach an event to the module as in section 6.3. Send the OFF event and the LED stops flashing. Power the module off and on again: the LED stays off. Send the ON event and the LED flashes again, and it carries on flashing after a power cycle.

A factory reset (the NNRSM opcode, or the push button sequence at power up) calls APP_factoryReset(), which turns blinking back on.